

Unidad II - Gramáticas Libres de Contexto

Lenguajes Formales y Computabilidad

Dra. Ana C. Olivera

FING - UNCuyo

Jerarquía de Gramáticas de Chomsky

Definición

Definiremos una *Gramática*, como una cuádrupla (V_n, V_t, S, P) donde:

- V_n es un conjunto finito de *símbolos no terminales* o simplemente no terminales.
- V_t es un conjunto finito de *símbolos terminales* o simplemente terminales.
- S es el *símbolo inicial* (no terminal).
- P es un conjunto finito de reglas de producción de la forma $\alpha \rightarrow \beta$.

Definición

Sea G una gramática descrita según la definición anterior podemos decir que:

- G es de tipo 0: Si la única restricción impuesta a sus reglas de producción es que $\alpha \in (V_n \cup V_t)^* \bullet V_n \bullet (V_n \cup V_t)^*$. Es decir al menos un *no terminal* en la parte izquierda de la regla de producción. A estas gramáticas las llamaremos *Gramáticas Estructuradas por Frases*.
- G se dice de tipo 1: Si $\alpha \in (V_n \cup V_t)^* \bullet V_n \bullet (V_n \cup V_t)^*$ y $\beta \in (V_n \cup V_t)^*$ y además cada regla verifica que $long(\alpha) \leq long(\beta)$. A estas gramáticas las llamaremos en el próximo capítulo *Sensibles al contexto*.
- G se dice de tipo 2: Si en cada regla $\alpha \rightarrow \beta$, $\alpha \in V_n$ y $\beta \in (V_n \cup V_t)^+$. Es decir la regla de producción tiene únicamente un no terminal del lado izquierdo de la gramática. A estas nos enfocaremos en este capítulo y las denominamos *Gramáticas Libres de Contexto*.
- G es de tipo 3: $\alpha \rightarrow \beta$, $\alpha \in V_n$ y $\beta \in (V_n \bullet V_t)$ o $\beta \in (V_t)$. Y en el capítulo anterior las llamamos *Gramáticas Regulares*.

Ejemplo

Supongamos que se desea obtener una gramática que permita generar todas las cadenas del lenguaje $L = \{wcw^R \mid w \in \{a, b\}^*\}$. La gramática $G = (V_n, V_t, S, P)$ tendrá la siguiente forma:

- $V_n = \{S\}$
- $V_t = \{a, b, c\}$
- $P = \{S \xrightarrow{(1)} aSa, S \xrightarrow{(2)} bSb, S \xrightarrow{(3)} c\}$

Esta gramática es de tipo 2 puesto que todas las reglas de producción tienen sus partes izquierdas $\in V_n$ y sus partes derechas $\in (V_n \cup V_t)^+$.

Ejemplo - Gramática Tipo 1

Ejemplo

Supongamos que se desea obtener una gramática sensible al contexto que permita generar todas las cadenas del lenguaje $L = \{a^n b^n c^n | n > 0\}$. La gramática $G = (V_n, V_t, S, P)$ tendrá la siguiente forma:

- $V_n = \{S, A, C\}$
- $V_t = \{a, b, c\}$
- $P = \left\{ \overbrace{S \rightarrow abc}^{(1)}, \overbrace{S \rightarrow aAbc}^{(2)}, \overbrace{A \rightarrow abC}^{(3)}, \overbrace{A \rightarrow aAbC}^{(4)}, \overbrace{Cb \rightarrow bC}^{(5)}, \right.$
 $\left. \overbrace{Cc \rightarrow cc}^{(6)} \right\}$

Sea $w = aaabbbccc$ generarla utilizando las reglas de G .

Ejemplo - Gramática Tipo 0

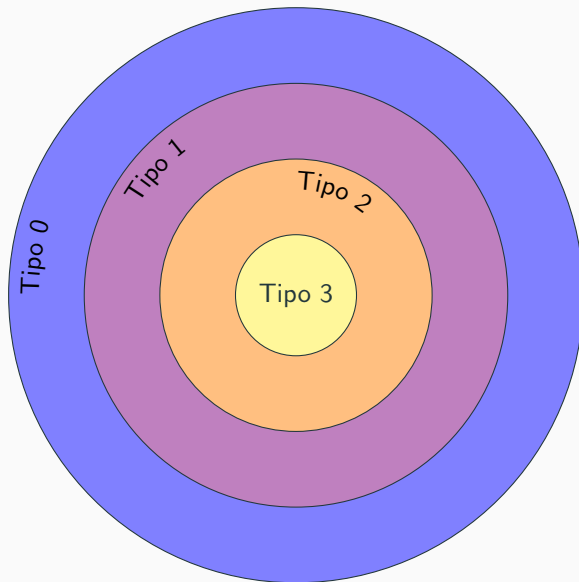
Ejemplo

Supongamos que se desea obtener una gramática que permita generar todas las cadenas del lenguaje $L = \{a^{2^n} | n \geq 0\}$. Una gramática que genera este lenguaje podría ser $G = (V_n, V_t, S, P)$ la cual tiene la siguiente forma:

- $V_n = \{S, X, Y, Z\}$
- $V_t = \{a\}$
- $P = \left\{ \overset{(1)}{S \rightarrow \lambda}, \overset{(2)}{S \rightarrow YXXY}, \overset{(3)}{YX \rightarrow YZ}, \overset{(4)}{ZX \rightarrow XZ}, \overset{(5)}{ZY \rightarrow XXXY}, \right.$
 $\left. \overset{(6)}{X \rightarrow a}, \overset{(7)}{Y \rightarrow \lambda} \right\}$

Esta gramática es de tipo 0 puesto que la $long(\alpha) \geq long(\beta)$ en las reglas $S \rightarrow \lambda$ e $Y \rightarrow \lambda$.

Sea $w = aaaaaa$ o sea $a^{2 \times 3}$ generarla utilizando las reglas de G .



Definición

Sea L un lenguaje, se dice que L es de tipo i si existe una gramática G de tipo i tal que $L(G) = L$ y además i es maximal en la jerarquía de Chomsky. Es decir, no existe G' de tipo d tal que $d > i$ y $L(G') = L(G)$.

Ejemplo - 2 gramáticas de distinto tipo para un mismo lenguaje

Ejemplo

Consideremos una gramática para generar el lenguaje $L = \{program\}$

- $G = (V_n, V_t, S, P)$
- $V_n = \{S, A, B\}$
- $V_t = \{p, r, o, g, r, a, m\}$
- $P = \{S \rightarrow AB, A \rightarrow pro, B \rightarrow gram\}$

Luego G es de tipo 2 ya que $S \rightarrow AB$ tiene dos no terminales del lado derecho de la regla de producción. Sin embargo es posible construir:

- $G' = (V_n, V_t, S, P)$
- $V_n = \{S\}$
- $V_t = \{p, r, o, g, a, m\}$
- $P = \{S \rightarrow program\}$

que es una gramática de tipo 3 o regular que también genera L . Luego se dice que L es regular ya que G' es la gramática de tipo mayor que genera a L en la Jerarquía de Chomsky.

Definiendo las Gramáticas Libres de Contexto

Definición

Una Gramática Libre de Contexto (GLC) es una cuádrupla $G = (V_n, V_t, S, P)$ tal que:

- V_n es un conjunto finito de símbolos *no terminales*.
- V_t es un conjunto finito de símbolos *terminales*.
- P es un conjunto finito de *reglas de producción* de la forma $\alpha \rightarrow \beta$ donde $\alpha \in V_n$ y $\beta \in (V_n \cup V_t)^+$

Ejemplo

Observemos la siguiente GLC G :

- $G = (V_n, V_t, O, P)$
- $V_n = \{O, A, S, V, F\}$
- $V_t = \{Javier, grande, verde, queso, come\}$
- $P = \{O \rightarrow FVF, F \rightarrow S, F \rightarrow SA, A \rightarrow grande, A \rightarrow verde, S \rightarrow queso, S \rightarrow Javier, V \rightarrow come\}$

Capacidades de las Gramáticas Libres de Contexto

Ejemplo

Sea $G = (V_n, V_t, S, P)$ donde:

- $V_n = \{S\}$
- $V_t = \{(\,,\,)\}$
- $P = \{S \xrightarrow{1} (), S \xrightarrow{2} SS, S \xrightarrow{3} (S)\}$

Observe atentamente la cadena $()(())$ es posible derivarla a través de dos derivaciones distintas comenzando por S .

$$S \xrightarrow{2} SS \xrightarrow{1} ()S \xrightarrow{3} ()(S) \xrightarrow{1} ()(())$$

$$S \xrightarrow{2} SS \xrightarrow{3} S(S) \xrightarrow{1} S(()) \xrightarrow{1} ()(())$$

Definición

Un árbol D es un *árbol de derivación* para una gramática $G = (V_n, V_t, S, P)$ si:

- La **raíz** del árbol es rotulada con S
- Cada **nodo** es rotulado con un símbolo de $V_n \cup V_t \cup \lambda$
- Si un nodo interior es rotulado con A y sus descendientes directos son rotulados $x_1, x_2, \dots, x_n$ implica que existe una producción en G de la forma $A \rightarrow x_1 x_2 \dots x_n$.

Los árboles de derivación constituyen un método visual de describir una derivación en una gramática libre de contexto.

Definición

A los nodos del árbol que no tienen descendientes se los denomina *hojas*. La *frontera* de un árbol de derivación es la cadena obtenida de la concatenación de los rótulos o etiquetas de las hojas de izquierda a derecha.

Ejemplo

Desarrolle un árbol de derivación de la cadena *aaabbb* en la gramática que genera $L = \{a^n b^n | n \geq 0\}$.

Teorema

Sea $G = (V_n, V_t, S, P)$ una GLC entonces para toda cadena $w \in L(G)$, $S \xrightarrow{G^+} w$ si y solo si existe un árbol de derivación de la gramática cuya frontera es w .

Demostración.

(ESQUEMA) Sea el conjunto de reglas de producción pertenecientes a P :

$$\begin{aligned} S &\rightarrow \underbrace{\dots}_{w_1} A_1 \underbrace{\dots}_{w'_1} \\ A_1 &\rightarrow \underbrace{\dots}_{w_2} A_2 \underbrace{\dots}_{w'_2} \\ &\vdots \\ A_{n-2} &\rightarrow \underbrace{\dots}_{w_{n-1}} A_{n-1} \underbrace{\dots}_{w'_{n-1}} \\ A_{n-1} &\rightarrow \underbrace{\dots}_{w_n} A_n \underbrace{\dots}_{w'_n} \end{aligned}$$

tales que permiten $S \xrightarrow{G^+} w_1 w_2 \dots w_{n-1} w_n w'_n w'_{n-1} \dots w'_2 w'_1$. En función de las reglas utilizadas en la derivación de la cadena podemos construir un árbol de derivación relacionado. Cada no terminal a la izquierda de las reglas utilizadas es la raíz de un sub-árbol cuyos hijos son los que están a la derecha de la regla en consideración. Para determinar la inversa, si se tiene un subárbol de derivación para una cadena $w = a_1 a_2 \dots a_{n-1} a_n$ es porque w constituye su frontera. Luego desde el árbol de derivación para la cadena pueden identificarse sub-árboles que generan subcadenas w y estos deben tener como raíz por definición a un no terminal. Luego, cada sub-árbol identificado se corresponde en sus hijos con la parte derecha de una regla de P y su raíz con la parte izquierda de esa misma regla P . Luego es posible generar dicha cadena a partir de

$P : S \xrightarrow{G^+} a_1 a_2 \dots a_{n-1} a_n$.

□

Ambigüedad

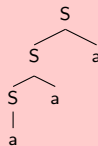
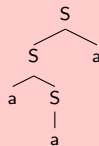
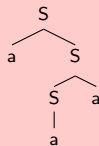
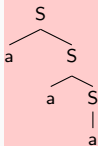
Consideremos la aparente inocua situación en que para una cadena existe más de un árbol de derivación posible.

Ejemplo

Sea $G = (V_n, V_t, S, P)$

- $V_n = \{S\}$
- $V_t = \{a\}$
- $P = \{S \xrightarrow{(1)} aS, S \xrightarrow{(2)} Sa, S \xrightarrow{(3)} a\}$

Podemos armar fácilmente cuatro árboles de derivación para la cadena aaa :



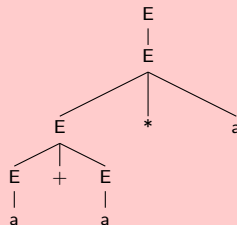
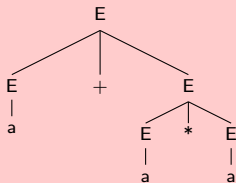
Veamos otro ejemplo.

Ejemplo

Sea $G = (V_n, V_t, E, P)$

- $V_n = \{E\}$
- $V_t = \{+, *, (,), a\}$
- $P = \{E \xrightarrow{(1)} E + E, E \xrightarrow{(2)} E * E, E \xrightarrow{(3)} (E), E \xrightarrow{(4)} a\}$

Consideremos la cadena $a + a * a$ podríamos armar por ejemplo dos árboles de derivación:



Definición

Se dice que una gramática G libre de contexto es *ambigua* si existe al menos una cadena $w \in L(G)$ para la cual existe más de un árbol de derivación con frontera w .

Ejemplo Gramática para Expresiones no ambigua

Ejemplo

Es posible, a veces, obtener otra gramática no ambigua. Por ejemplo, sea $G = (V_n, V_t, E, P)$ donde:

- $V_n = \{T, F, E\}$
- $V_t = \{+, *, (,), id\}$
-

$$P = \{ E \xrightarrow{(1)} E + T, \\ E \xrightarrow{(2)} T, \\ T \xrightarrow{(3)} T * F, \\ T \xrightarrow{(4)} F, \\ F \xrightarrow{(5)} (E), \\ F \xrightarrow{(6)} a \}.$$

No es ambigua

Ejemplo - Ambigüedad en lenguajes de programación

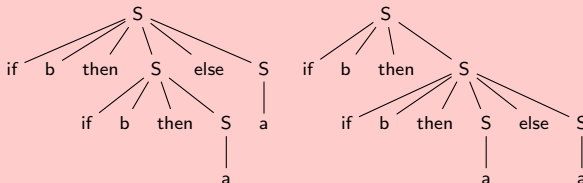
Ejemplo

Uno de los ejemplos más conocidos en la literatura sobre ambigüedad de un lenguaje de programación imperativo es Pascal y su *else* pendiente (*dangling else* en inglés). Considere la siguiente gramática $G = (V_n, V_t, S, P)$:

- $V_n = \{S\}$
- $V_t = \{if, then, else, a, b\}$
- $P = \{S \xrightarrow{(1)} if\ b\ then\ S\ else\ S, S \xrightarrow{(2)} if\ b\ then\ S, S \xrightarrow{(3)} a\}$

Donde a representa las sentencias del lenguaje Pascal y b las sentencias booleanas válidas en Pascal.

Observe atentamente que G es ambigua ya que la sentencia *if b then if b then a else a* tiene dos árboles de derivación. Entonces se pueden dar dos interpretaciones distintas sobre el *if* al cual corresponde el *else*. Es decir, según como armemos el árbol de derivación, el *else* puede corresponder al primer *if* o al segundo.



Ejemplo

En general, la ambigüedad se resuelve con alguna convención, por ejemplo, que el *else* corresponde con el último *then*. Por ejemplo, el famoso Pascal marca con *begin-end* el comienzo y el fin de una sentencia compuesta.

Entonces tomemos la gramática y hagamos cambios para evitar la ambigüedad. Una forma es agregando no terminales. Usaremos dos nuevos no terminales, uno que genere construcciones de la forma *if then else* y el otro que genere tanto *if then* como *if then*. Sea $G = (V_n, V_t, S_1, P)$:

- $V_n = \{S_1, S_2\}$
- $V_t = \{if, then, else, a, b\}$
-

$$\begin{aligned} P = \{ & S_1 \xrightarrow{(1)} if\ b\ then\ S_1, \\ & S_1 \xrightarrow{(2)} if\ b\ then\ S_2\ else\ S_1, \\ & S_1 \xrightarrow{(3)} a, \\ & S_2 \xrightarrow{(4)} if\ b\ then\ S_2\ else\ S_2, \\ & S_2 \xrightarrow{(5)} a \} \end{aligned}$$

El hecho de que solo S_2 precede al *else* asegura que entre el par *then-else* hay sentencias simples o con *then-else* entonces se evita la ambigüedad.

Definición

Se dice que un lenguaje L libre de contexto es *inherentemente ambiguo* si toda gramática G que genera L es ambigua. O dicho de otra manera no existe una gramática G no ambigua que genere L .

Ejemplo

Tomemos el LLC $L = \{a^i b^j c^k \mid i = j \text{ o } j = k, i, j, k \geq 1\}$. L es un lenguaje inherentemente ambiguo.

- Sipser Sección 2.1
- Lewis Secciones 3.1 3.2
- Apunte cátedra